



**Centrul Județean de
Exelență Vaslui**

I⁰⁰¹⁰Info⁰⁰⁰⁰ Educație
Focșani - 2025
Olimpiada Națională de Inovare și Creativitate Digitală



T.A.R.S.I.S.

**Tehnologie Avansată pentru Relații și Suport în
Interacțiuni Sociale**

Secțiune: Științe Aplicate

Categorie: Seniori

Elevi:

- Mereuță Robert - Software
- Chirițescu Denis - Hardware

Coordonator:

- Chirițescu Ana Carmen



Scop

TARSIS(Tehnologie Avansată pentru Relații și Suport în Interacțiuni Sociale) este o platformă robotică umanoidă cu funcție de asistent vocal utilizat în spații unde este importantă interacțiunea socială, de exemplu recepții, spitale, școli etc. Sistemul robotic poate lucra în zone care necesită asistență umană, astfel este evidențiată importanța inteligenței artificiale și a roboticii în lumea oamenilor. De exemplu, TARSIS poate oferi suport în instituții din domeniul medicinei, precum centre medicale destinate persoanelor cu dizabilități. Robotul a fost proiectat să funcționeze atât în domenii tehnice, cât și în cele non-tehnice (interacțiuni sociale), având capacitatea de a răspunde la întrebări din domenii vaste, cum ar fi matematică, fizică, programare, biologie și multe altele. TARSIS poate fi folosit și în domeniul STEM (Știință, Tehnologie, Inginerie și Matematică), de exemplu, în cadrul unor evenimente de tip workshop organizate într-un club de robotică poate fi implicat în procesul educational. Problema pe care TARSIS o poate rezolva este lipsa de resursă umană în anumite instituții care necesită interacțiune socială constantă și suport pentru persoanele aflate în nevoie.

Una din funcțiile de bază ale robotului este de asigurare a asistenței și uneori înlocuire a personalului din instituțiile suprasolicitate cu personal puțin și unde este nevoie de executare a unor sarcini repetitive pentru deservirea pacienților sau clienților.

Obiective

1 Folosirea inteligenței artificiale în interacțiuni sociale;

3 Optimizarea proceselor de comunicare în tratarea pacienților;

2 Automatizarea și optimizarea proceselor repetitive;

4 Studiul utilizării diferitelor tipuri de inteligență artificială pe arhitecturi hardware de tip Raspberry PI și Google Coral.



Componente

Corp:

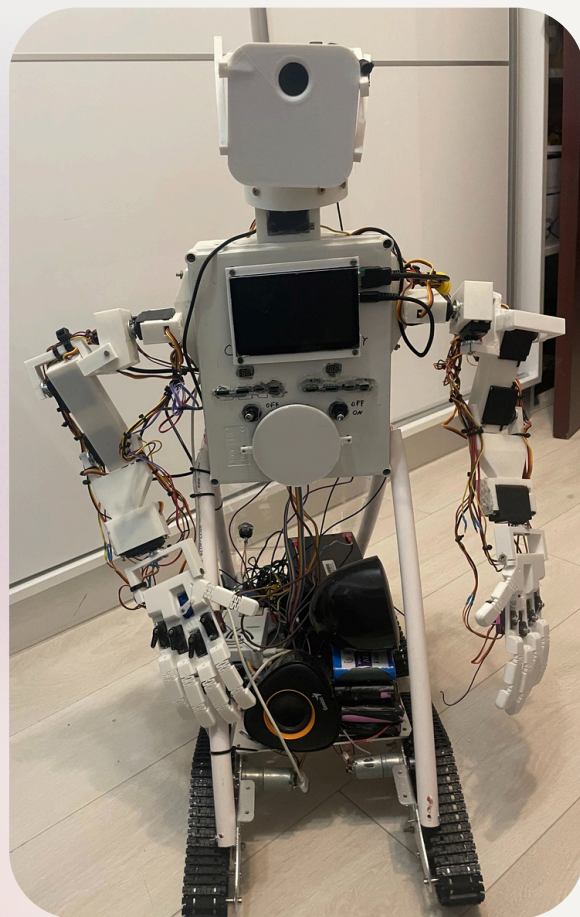
Brațe : Pe robot sunt montate două brațe, fiecare având câte cinci grade de libertate reprezentate de servomotoare MG996R Tower Pro. De asemenea, fiecare deget este acționat de câte un servomotor MG90S folosind fir de nylon. Fiecare servomotor este montat într-o carcasă realizată prin imprimare 3D, care include câte un compartiment destinat unui rulment, care optimizează mișcarea de rotație pe componentă mecanică.

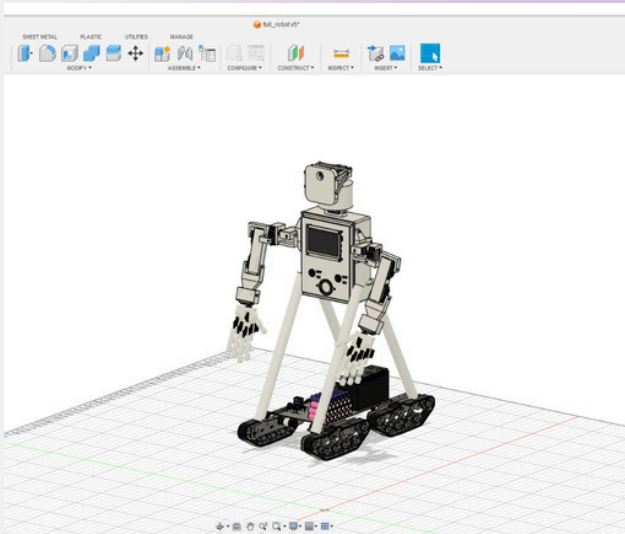
Cap: Dispune de două grade de libertate, fiecare fiind controlat de câte un servomotor MG996R. Mecanismul de fixare a feței, precum și mecanismele de rotație a componentelor sunt imprimate 3D. În spatele feței este integrată o cameră web cu rezoluție 2K Ultra-HD, destinată recunoașterii vizuale a obiectelor și a persoanelor. Este montat un microfon de tip lavalieră pentru înregistrarea vocii utilizatorului;

Trunchiul: Construit dintr-o cutie din material plastic, compartimentul intern conține un SBC(Single Board Computer) de tip Raspberry Pi 4 în care sunt conectate 2 drivere PCA9685 pentru servomotoare și un convertor coborâtor de tensiune "Buck". Primul driver controlează servomotoarele de la mâna stângă și de la cap, iar al doilea controlează motoarele de la mâna dreaptă. Pe suprafața externă este plasat un ecran tactil conectat la Raspberry Pi, două încărcătoare pentru baterii Li-Ion fiecare având câte un buton de tip switch pentru a porni / opri curentul de la microcontroller și de la brațe. Este plasat și un inel Neopixel cu 24 de LED-uri pentru a oferi feedback vizual utilizatorilor.

Bază (Șasiu):

Șasiu: Este construit din aluminiu, asigurând o structură rezistentă. Pe partea posterioară sunt montate patru motoare DC, fiecare acționând câte un sistem de șenile, fiind prevăzute cu suspensii bazate pe arcuri pentru a permite adaptarea pe diferite tipuri de teren. Pe partea superioară sunt montate două drivere de motoare DC (L298N) alimentate de un set de acumulatori Li-Ion la 12V. Al doilea SBC Raspberry Pi 4 este responsabil pentru gestionarea vocii utilizatorului și generarea de răspunsuri, este alimentat de o baterie externă. Brațele sunt alimentate de un acumulator pe bază de plumb acid la 12V montat pe șasiu.

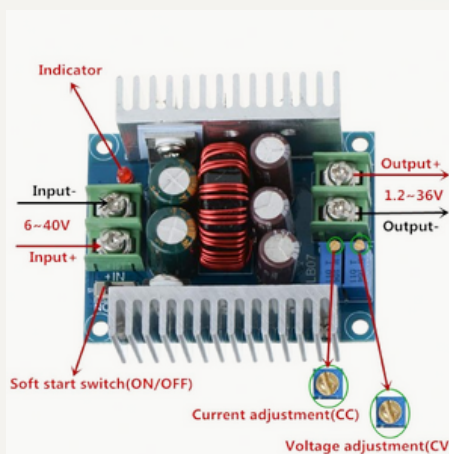




Componentele brațelor și ale capului au fost proiectate folosind programul Fusion 360, dezvoltat de Autodesk, acest program permite modelarea și proiectarea componentelor în plan 3D. Pentru a preveni erorile de funcționare au fost generate simulări și animații. Rotația articulațiilor a fost creată folosind funcția "Joint", care creează articulații, componentele au o articulație cu mișcare de rotație, iar motoarele sunt prinse de componente cu o articulație solidă. Folosind Fusion 360 pot fi realizate simulări legate de proprietățile materialelor, randări ale modelelor și multe altele.

Alimentare și gestionarea

consumului de curent



Pentru a alimenta placa SBC Raspberry Pi 4, folosită la procesarea imaginilor și recunoașterea obiectelor, a fost realizată o sursă de alimentare alcătuită din trei pachete de acumulatori Li-Ion, fiecare conținând șapte acumulatori individuali.

Fiecare acumulator are o tensiune standard de 3.6V și o capacitate de 1200mAh. Celulele sunt conectate în paralel asigurând o tensiune maximă de maxim 4.2V și o capacitate totală de aproximativ 25.000mAh. Sistemul folosește un modul dedicat pentru încărcarea acumulatorilor Li-Ion, echipat cu mai multe porturi USB prin care este alimentată placa Raspberry Pi, dar pot fi încărcăți acumulatorii folosind cablu USB. Brațele sunt alimentate separat, deoarece motoarele consumă în jur de 4A când sunt în funcțiune și aproximativ 2A în repaus. Acumulatorul folosit este de tip plumb-acid, cu o tensiune standard de 12V și o capacitate de 12Ah.



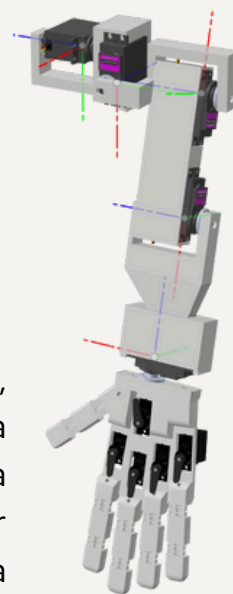
Deoarece motoarele funcționează între 4,8V și 6,6V a fost adăugat un convertor buck de care coboară tensiunea la 8V. Motoarele șasiului sunt alimentate de un pachet de baterii Li-Ion la 12V, format din cinci acumulatori reîncărcabili, asigurând o sursă de energie stabilă pentru a pune în mișcare sistemul de șenile. Modulul Raspberry Pi 4, montat pe șasiu, este alimentat dintr-o baterie externă de 5V și 20.000mAh.

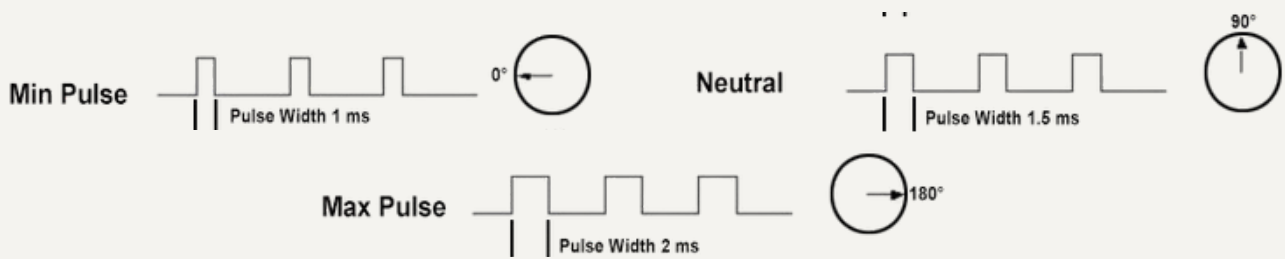
Deoarece motoarele funcționează între 4,8V și 6,6V a fost adăugat un convertor "buck" care coboară tensiunea la 8V. Motoarele șasiului sunt alimentate de un pachet de baterii Li-Ion la 12V, format din cinci acumulatori reîncărcabili, asigurând o sursă de energie stabilă pentru a pune în mișcare sistemul de șenile. Modulul Raspberry Pi 4, montat pe șasiu, este alimentat dintr-o baterie externă de 5V și 20.000mAh.

Algoritmul de control al servomotoarelor și determinarea coordonatelor în spațiu tridimensional

Servomotoarele sunt controlate folosind două drivere de tip PCA9685, fiecare driver are 16 canale disponibile pentru servomotoare. Conexiunea între Raspberry Pi 4 și drivere se face folosind protocolul I2C, iar pentru a putea utiliza ambele module au fost modificate adresele I2C. Un driver are adresa 0x41 și controlează brațul stâng și capul, iar celălalt are adresa 0x42 și controlează brațul drept. Servomotoarele sunt acționate prin semnal PWM(Pulse Width Modulation) unghiurile de rotație fiind controlate de lățimea semnalului. Algoritmul de control a fost implementat folosind limbajul de programare Python. Valoarea unghiului de rotație a servomotorului este convertită în lățime de semnal PWM folosind formula:

$$pulse_{length} = \left(\frac{angle}{180} \right) \cdot (servo_{max} - servo_{min}) + servo_{min}$$





Pentru a determina coordonatele precise ale vârfului unui braț robotic este folosită cinematica directă, care se bazează pe utilizarea matricilor de rotație, fiecare articulație adaugă o transformare de rotație asupra poziției capătului brațului. Vom utiliza matricile de rotație pe fiecare axă, fiind utilizate doar axele X și Y.

Pentru rotirea punctelor în jurul axei X va fi folosită matricea:

$$R_X(\theta_X) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_X & -\sin \theta_X \\ 0 & \sin \theta_X & \cos \theta_X \end{pmatrix}$$

Pentru rotirea punctelor în jurul axei Y va fi folosită matricea:

$$R_Y(\theta_Y) = \begin{pmatrix} \cos \theta_Y & 0 & \sin \theta_Y \\ 0 & 1 & 0 \\ -\sin \theta_Y & 0 & \cos \theta_Y \end{pmatrix}$$

Poziția finală a capătului brațului se calculează prin aplicarea a matricelor de rotație și adăugarea translațiilor pe fiecare segment, luând în considerare mișcarea articulațiilor anterioare. Poziția se calculează prin aplicarea matricilor de rotație, poziția finală a capătului segmentului este obținută astfel (fără a fi influențată de restul articulațiilor):

$$P_n = P_{n-1} + R_X(\theta_{X_n}) \cdot R_Y(\theta_{Y_n}) \cdot D_n$$

unde este vectorul de translație cu valoarea: $D_n = \begin{pmatrix} L_n \\ 0 \\ 0 \end{pmatrix}$

dacă mișcarea are loc pe axa X și $D_n = \begin{pmatrix} 0 \\ L_n \\ 0 \end{pmatrix}$ dacă mișcarea are loc pe axa Y.

Având în vedere faptul că brațul se mișcă în mare parte doar pe axele X și Y, vectorul pentru axa Z nu va fi folosit.

Poziția capătului brațului $P_n = \begin{pmatrix} x_n \\ y_n \\ z_n \end{pmatrix}$ poate fi obținută succesiv aplicând rotațiile pentru toate articulațiile.

Formula generală pentru a determina poziția finală a brațului luând în considerare toate articulațiile:

$$P_n = \sum_{k=1}^n (R_x(\theta_{x_k}) \cdot R_y(\theta_{y_k}) \cdot D_k)$$

Această sumă reține efectele succesive ale fiecărei rotații asupra poziției finale a capătului brațului. Rotațiile sunt efectuate în ordinea articulațiilor, iar fiecare segment adaugă o transformare adițională la poziția finală a capătului brațului.

Utilizarea Inteligenței Artificiale pentru sinteza vocală și generarea de răspunsuri

Programul utilizat pentru sinteza vocală și pentru a genera răspunsuri pe Raspberry Pi 4 a fost dezvoltat folosind limbajul de programare Python și este adaptat pentru a comunica în limba română. Biblioteca "speech_recognition" folosește serviciul Google Web Speech API, o platformă bazată pe Google Cloud utilizând inteligență artificială, care permite recunoașterea și conversia vocii umane în text. Semnalul audio captat de microfon este procesat și trimis la serverele Google și apoi este procesat din nou folosind algoritmi de Machine Learning și Deep Learning. Dacă recunoașterea vocală este realizată cu succes funcția "takeCommand()" returnează textul obținut care poate fi utilizat pentru a executa comenzi predefinite sau pentru a răspunde la întrebarea utilizatorului. Astfel interacțiunea cu robotul poate fi realizată prin comenzi vocale, oferă o experiență interactivă și prietenoasă în medii care necesită interacțiune socială.

Programul intră într-o buclă principală care continuă să ruleze la infinit, utilizatorul poate opri tot programul prin comanda "stop". În cazul în care utilizatorul nu oferă o comandă predefinită, de exemplu "ridică mâna", "Salut" sau "Stop", programul apelează funcția "query_chatgpt()". Funcția folosește API-ul (Application Programming Interface) de la ChatGPT versiunea "gpt-4o". ChatGPT este un chatbot baza pe inteligență artificială utilizat la a generarea răspunsurilor la orice întrebare. A fost adăugată o limită de 50 de cuvinte la fiecare răspuns generat, astfel răspunsul este destul de scurt și direct. Răspunsurile sunt generate instantaneu, iar pentru a îmbunătăți interacțiunea cu utilizatorul sunt utilizate LED-urile amplasate pe partea frontală a trunchiului oferind un feedback vizual având și funcția de „progress bar”. Atunci când este inițializată ascultarea comenzii cercul de LED-uri devine albastru, se schimbă în roșu atunci când se termină secvența, iar la generarea răspunsului acesta devine albastru deschis variind intensitatea luminii.

Procesul de TTS (Text to Speech) reprezintă conversia textului într-o formă vocală prin utilizarea modulului "pyttsx3", care permite configurarea unor parametri, cum ar fi vocea și/sau viteza de vorbire. Funcția "speak()" trimite textul către motorul TTS care se ocupă de conversia acestuia în semnal audio.

Este setat cuvântul "Eugen" ca și wake-up word, desemnând astfel numele robotului umanoid.

Acest mecanism creează o interacțiune mai naturală și prietenoasă, contribuind la îmbunătățirea relației sociale dintre utilizator și robot.

```
def takeCommand():
    r = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening...")
        r.adjust_for_ambient_noise(source, duration=1)
        r.pause_threshold = 1
        indicate_listening()
        audio = r.listen(source)

    try:
        print("Recognizing...")
        query = r.recognize_google(audio, language='ro-RO')
        print(f"User said: {query}")
    except Exception as e:
        print(f"An error occurred: {e}")
        query = ""
    stop_listening_signal()
    return query
```

```
elif "stângă" in query and "ridică" in query or "stanga" in query and "ridică" in query:
    send_message_command("stanga")
    time.sleep(3)
elif "dreapta" in query and "ridică" in query or "dreapta" in query and "ridică" in query:
    send_message_command("dreapta")
    time.sleep(3)
else:
    process = subprocess.Popen(["sudo", "/home/robert/venv/bin/python3", "response.py"])
    response_text = query_chatgpt(query)
    if response_text:
        print(response_text)
        speak(response_text)
    process.terminate()
sys.exit(app.exec_())
```

```
def query_chatgpt(user_message, word_limit=50):
    limited_prompt = (
        f"Răspunde la următoarea întrebare folosind cel mult {word_limit} cuvinte."
        f"Tu te numești Eugen"
    )
    messages.append({"role": "user", "content": f"{limited_prompt}\n{user_message}"})

    try:
        chat = openai.ChatCompletion.create(
            model="gpt-4o",
            messages=messages,
            timeout=10
        )
        reply = chat.choices[0].message.content.strip()
        messages.append({"role": "assistant", "content": reply})
        return reply
    except openai.error.OpenAIError as e:
        print(f"Eroare OpenAI: {e}")
        return "Îmi pare rău, dar nu pot răspunde acum."
```

Procesarea imaginilor și recunoașterea obiectelor prin utilizarea IA

Procesarea imaginilor este realizată de modulul de procesare Raspberry Pi 4 împreună cu un procesor extern Google Coral Edge TPU, cu rol de a accelera procesele de IA. Procesorul extern Google Coral Edge TPU rulează procesele de IA din program, funcțiile de procesare ale imaginilor și funcțiile de mișcare sunt îndeplinite de procesorul ARM Cortex-A72 de pe Raspberry Pi 4.

Puterea de procesare a modului accelerator este de 4×10^{12} operații pe secundă la 0.5W asigurând recunoașterea obiectelor la 60fps (frames per second) în loc de 10 (procesate cu CPU de la Raspberry). Procesarea imaginilor folosind Inteligența Artificială este structurată în program prin două funcții, funcția "corel_detect()" unde sunt încărcate fișierele pentru modelul TensorFlow, fișierul cu numele obiectelor și alți parametri. Funcția face un „parser” de argumente pe care le adaugă automat în linia de comandă atunci când este rulat programul, argumente precum locația modelului, locația fișierului de nume, numărul de detecții prioritare și filtrul de probabilitate. De asemenea este înregistrată imaginea de pe cameră și este procesată folosind acceleratorul de IA. Cealaltă funcție, numită "append_objs_to_img()", are patru parametri: imaginea originală luată de la cameră, dimensiunea imaginii, lista obiectelor și ID-urile numelor obiectelor.

Funcția scalează imaginea, desenează pătrate în jurul fiecărui obiect detectat, afișează probabilitatea ca obiectul respectiv să fie același cu predicția și afișează denumirea obiectului. Imaginea este afișată pe ecran și sunt adăugate dreptunghiurile de detecție, denumirea obiectului identificat și valoarea potrivirii între obiect și setul de date.



```
def coral_detect():
    default_model_dir = '../all_models'
    default_model = 'mobilenet_ssd_v2_coco_quant_postprocess_edgetpu.tflite'
    default_labels = 'coco_labels.txt'

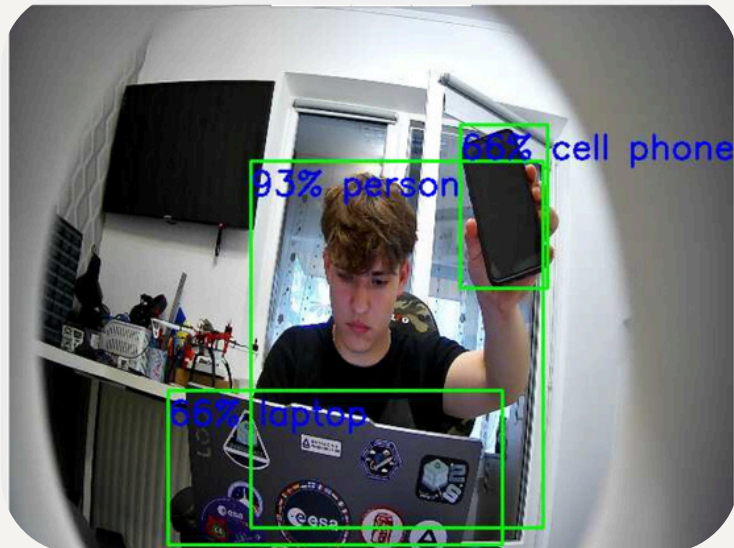
    parser = argparse.ArgumentParser()
    parser.add_argument('--model', default=os.path.join(default_model_dir, default_model))
    parser.add_argument('--labels', default=os.path.join(default_model_dir, default_labels))
    parser.add_argument('--top_k', type=int, default=3)
    parser.add_argument('--camera_idx', type=int, default=0)
    parser.add_argument('--threshold', type=float, default=0.1)
    args = parser.parse_args()

    print(f'Loading model: {args.model} with labels: {args.labels}')
    interpreter = make_interpreter(args.model)
    interpreter.allocate_tensors()
    labels = read_label_file(args.labels)
    inference_size = input_size(interpreter)

    cap = cv2.VideoCapture(args.camera_idx)
    cv2.namedWindow('Object Detection', cv2.WND_PROP_FULLSCREEN)
    cv2.setWindowProperty('Object Detection', cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        cv2_im_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        cv2_im_rgb = cv2.resize(cv2_im_rgb, inference_size)
        run_inference(interpreter, cv2_im_rgb.tobytes())
        objs = get_objects(interpreter, args.threshold)[args.top_k]
```



Algoritmul de adaptare al comportamentului și al emoțiilor

Având în vedere că API-ul de ChatGPT utilizat pentru a genera răspunsuri nu are inclusă memoria conversațiilor, am inclus în program un fișier ".json" unde este reținut istoricul mesajelor. Conținutul fișierului este de forma:

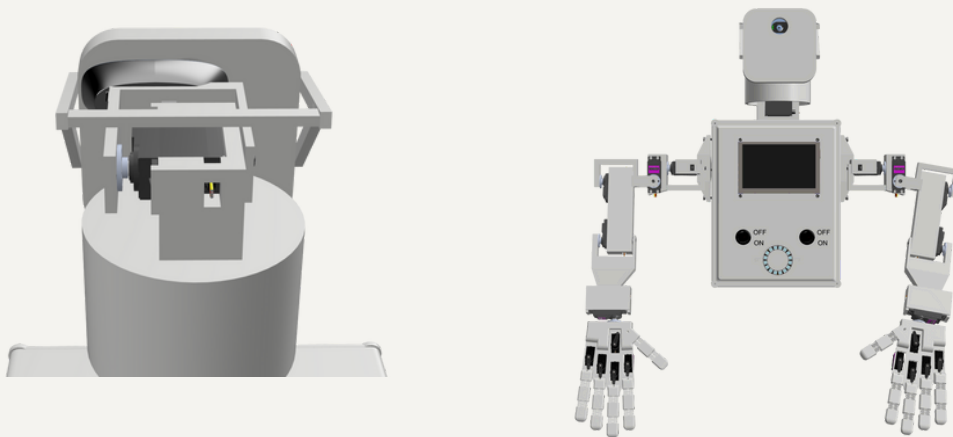
```
{
  "role": "assistant sau user",
  "content": "conținut"
}
```

Atunci când utilizatorul trimite un mesaj (o conversație), sistemul adaugă la acest mesaj un text predefinit cu scopul de a adăuga informații despre platforma hardware, numele robotului, procedura de răspuns întrebărilor utilizatorului. Motorul de Inteligență Artificială este setat să răspundă într-un mod empatic și să observe modificarea comportamentului utilizatorului din istoricul mesajelor. La pornirea programului fișierul de memorie a conversației este golit, iar în el sunt stocate doar ultimele trei mesaje din conversație, pentru a nu încărca limita de introducere al prompt-ului. Astfel, platforma robotică umanoidă poate simula anumite emoții în interacțiunea cu utilizatori integrându-se în rândul oamenilor.

Mecanismele mecanice utilizate în membrele și articulațiile biomimetice

Capul a fost proiectat pentru a imita structura unui cap uman utilizând două axe de rotație și include o camera 2KUHD. Mecanismul de rotație pe axa X este compus dintr-un servomotor, căruia îi este atașat un cilindru, pe care sunt montate camera și microfonul.

Brațele biomimetice sunt constituite din cinci articulații (grade de libertate) imitând structura brațelor umane și conferind robotului o formă umanoidă. Fiecare articulație este echipată cu un servomotor montat într-o carcasă proiectată special pentru a acomoda acest tip de motor, iar la extremitatea opusă axei de rotație este integrat un rulment cu diametru interior de 3mm. Pentru a conecta o componentă la un servomotor, se adăugă o zonă destinată unei piulițe M3, care se potrivește cu gaura interioară a rulmentului, permițând astfel introducerea unui șurub prin piuliță în rulment. Acest mecanism asigură transmiterea mișcării de rotație de la servomotor către componenta responsabilă de rotirea următoarei articulații.



Structura programelor și modul de operare a platformei robotice

Cele două programe sunt lansate automat la pornirea modulelor Raspberry Pi 4 și funcționează independent. Programul rulat pe SBC-ul responsabil pentru gestionarea răspunsurilor începe cu o secvență de încărcare în cadrul căreia cercul de LED-uri se aprinde secvențial, fiecare LED având o culoare diferită. În paralel al doilea program se ocupă cu prelucrarea și afișarea imaginilor capturate de cameră, integrând și detecția obiectelor. Începe să asculte pe un canal socket pentru a primi date externe legate de controlul membrelor.

În momentul în care programul responsabil de gestionarea răspunsurilor primește numele utilizatorului robotul va saluta utilizatorul printr-un gest al mâinii drepte. După această interacțiune inițială se activează bucla principală (main loop) pe ambele plăci permițând utilizatorului să interacționeze cu sistemul, apelând diverse comenzi prestabilite sau inițiind o conversație cu ajutorul motorului de inteligență artificială.

În cazul în care utilizatorul solicită utilizarea membrilor biomimetice printr-o comandă vocală pre-definită („ridica mana”), programul de procesare a vocii va recunoaște comanda și apoi va transmite o comandă către procesorul responsabil de mișcarea membrilor robotului, activând membrele respective în funcție de cerințele utilizatorului.

Conceptele și provocările tehnice întâmpinate

Inițial, în locul șasiului de aluminiu proiectul preconiza utilizarea unui picior biomimetic dotat cu trei articulații acționate prin intermediul actuatorilor. Componentele acestui picior sunt alcătuite din coapsă, tibie și laba piciorului, fiind conectate între ele prin intermediul axurilor hexagonale. Fiecare dintre aceste componente dispune de două urechi de prindere destinate pentru fixarea actuatorilor, asigurând astfel mobilitatea și funcționalitatea necesare pentru imitația corectă a unui picior uman.

Principalele provocări tehnice au apărut în gestionarea curentului electric și a consumului de energie. Inițial, pentru alimentarea servomotoarelor, a fost utilizat un pachet de acumulatori Li-Ion format din zece celule. Cu toate acestea, s-a observat că aceștia se descărcău rapid și că tensiunea furnizată era prea mică pentru a controla motoarele la viteze mari, ceea ce a condus la o capacitate de ridicare a unei greutăți mai scăzută. Ulterior, s-a optat pentru utilizarea unui acumulator pe bază de plumb-acid, pentru a asigura o sursă de energie mai stabilă și adecvată pentru operarea servomotoarelor la parametri optimi.

Concluzii

1. Implementarea proiectului TARSIS:

Sistemul a fost proiectat pentru a îmbunătăți interacțiunile într-o gamă largă de domenii și instituții (școli, spitale, unități medicale pentru persoane cu dizabilități), demonstrând potențialul soluțiilor automatizate.

2. Automatizarea și eficientizarea proceselor:

TARSIS poate îmbunătăți procesul de comunicare socială, contribuind la optimizarea timpului și gestionarea resurselor umane din instituții, reducând sarcinile personalului uman.

3. Impactul educativ în domeniul STEM:

TARSIS reprezintă un exemplu de utilizare a roboticii și a Inteligenței Artificiale în educația STEM, susținând învățarea prin furnizarea unor largi biblioteci informaționale în cadrul activităților educaționale.

4. Posibilități de extindere în viitor.

Posibilitățile de personalizare a robotului TARSIS permit o implementare flexibilă în foarte multe domenii.

Bibliografie

- [§https://cdn-learn.adafruit.com/downloads/pdf/16-channel-pwm-servo-driver.pdf](https://cdn-learn.adafruit.com/downloads/pdf/16-channel-pwm-servo-driver.pdf)
- [§https://users.cs.duke.edu/~brd/Teaching/Bio/asmb/current/Papers/chap3-forward-kinematics.pdf](https://users.cs.duke.edu/~brd/Teaching/Bio/asmb/current/Papers/chap3-forward-kinematics.pdf)
- [§https://www.rosroboticslearning.com/forward-kinematics](https://www.rosroboticslearning.com/forward-kinematics)
- [§http://arhiva.mec.upt.ro/fisiere/2021-2022/Lic_Dis/Biblio/Tematica%20%C8%99i%20bibliografia%20licei%C8%9B%C4%83%20Mecatronic+Robotic%C4%83%202022/BR/Capitol%202.pdf](http://arhiva.mec.upt.ro/fisiere/2021-2022/Lic_Dis/Biblio/Tematica%20%C8%99i%20bibliografia%20licei%C8%9B%C4%83%20Mecatronic+Robotic%C4%83%202022/BR/Capitol%202.pdf)
- [§https://www.instructables.com/ASPIR-Full-Size-3D-Printed-Humanoid-Robot/](https://www.instructables.com/ASPIR-Full-Size-3D-Printed-Humanoid-Robot/)
- [§https://github.com/chuanqi305/MobileNet-SSD](https://github.com/chuanqi305/MobileNet-SSD)
- [§https://pypi.org/project/SpeechRecognition/](https://pypi.org/project/SpeechRecognition/)
- [§https://www.figure.ai/](https://www.figure.ai/)
- [§https://www.analyticsvidhya.com/blog/2022/04/object-detection-using-haar-cascade-](https://www.analyticsvidhya.com/blog/2022/04/object-detection-using-haar-cascade-)
- [§https://eloquentcode.com/synthesize-speech-in-a-different-language-using-python](https://eloquentcode.com/synthesize-speech-in-a-different-language-using-python)
- [§https://nibmehub.com/opac-service/pdf/read/Theory%20of%20Applied%20Robotics%20Kinematics-%20Dynamics-%20and%20Control.pdf](https://nibmehub.com/opac-service/pdf/read/Theory%20of%20Applied%20Robotics%20Kinematics-%20Dynamics-%20and%20Control.pdf)
- §John D. Kelleher, "Deep Learning", MIT Press 2019
- John Mueller, Luca Massaron, "Deep Learning for Dummies", 2019